UE24CS242B : Operating Systems

ORANGE-2

NAME: Qamar Ahmed

SRN: PES2UG24CS904

SEC: D

GITHUB: https://github.com/qamarahmoodi/PES2UG24CS904-pes-vcs

PHASE 1:

1A:

```
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$
```

1B:

```
qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$
```

PHASE 2:

2A:

```
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$
```

2B:

```
qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$ find .pes/objects -type f
.pes/objects/37/1aac45c3ba401d13c2ce98c9537b4e53c6049b950bf8a33cbc400712d9f4fa
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/5d/7c77a2e6a271ba3d3d3093913af347e0f850856d8548f7eab9522e8aa1382e
qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$
```

2C:

```
qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$ xxd .pes/objects/37/1aac45c3ba401d13c2ce98c9537b4e53c6049b950bf8a
33cbc400712d9f4fa | head -20
00000000: 7472 6565 2034 3500 3130 3036 3434 2061  tree 45.100644 a
00000010: 2e74 7874 002c f8d8 3d9e e295 43b3 4a87  .txt.,..=...C.J.
00000020: 7274 21fd ecb7 e3f3 a183 d337 6390 25de  rt!........7c.%.
00000030: 576d b9eb b4                             Wm...
qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$
```

PHASE 3:

3A:

```
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  tree.c
  untracked:  pes.c
  untracked:  index.h
  untracked:  README.md
  untracked:  test_tree.c
  untracked:  test_sequence.sh
  untracked:  pes.h
  untracked:  a.txt
  untracked:  tree.h
  untracked:  index.c
  untracked:  commit.h
  untracked:  object.c
  untracked:  commit.c
  untracked:  Makefile

qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$
```

3B:

```
qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776444787 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776444787 6 file2.txt
qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$
```

PHASE 4:

4A:

```
Committed: f945e7e61304... Initial commit
Committed: fa3d22a564f7... Add world
Committed: 0c3f4dd8082e... Add farewell
commit 0c3f4dd8082e8b67ac96c86e32a2f0a79724a6b9bdd1eb2e8d6e0abb87dd258d
Author: PES User <pes@localhost>
Date:   1776444836

    Add farewelldd f

commit fa3d22a564f7f2539d2baf66fbc4862b2b5bb02c6eca8d32ad947f005852be96
Author: PES User <pes@localhost>
Date:   1776444835

    Add worldelldd f

commit f945e7e6130460526d9bf765f6d56d336049828125b50e8146af76a61ba11e9b
Author: PES User <pes@localhost>
Date:   1776444835

    Initial commit

qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$
```

4B:

```
qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/0c/3f4dd8082e8b67ac96c86e32a2f0a79724a6b9bdd1eb2e8d6e0abb87dd258d
.pes/objects/48/1407f7237f7bd749dca31e2d8422bafe211741d5556f007e0faf821ffc7179
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/f3/97707e14d1cabf1c29a72d833bffb94d4a474ecc48789d6496742ae6541034
.pes/objects/f9/45e7e6130460526d9bf765f6d56d336049828125b50e8146af76a61ba11e9b
.pes/objects/fa/3d22a564f7f2539d2baf66fbc4862b2b5bb02c6eca8d32ad947f005852be96
.pes/refs/heads/main
qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$
```

4C:

```
qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
0c3f4dd8082e8b67ac96c86e32a2f0a79724a6b9bdd1eb2e8d6e0abb87dd258d
ref: refs/heads/main
qamar@qamar-VirtualBox:~/PES2UG24CS904-pes-vcs$
```

PHASE 5 AND 6:

Q5.1 — How would you implement pes checkout <branch>?:

To implement pes checkout , we need to update both the reference files and the working directory.

First, we check if the branch exists in .pes/refs/heads/. If it exists, we update the .pes/HEAD file to point to that branch (for example: ref: refs/heads/branch_name).

Next, we read the commit hash stored in that branch file. Then we load the tree corresponding to that commit and update the working directory to match it. This means creating new files, modifying existing ones, and deleting extra files.

This operation is complex because we must handle cases where the user has uncommitted changes. We also need to correctly recreate directory structure and file modes from the tree.

Q5.2 — How to detect dirty working directory?:

To detect a dirty working directory, we compare the current files with the index.

For each file in the index, we use stat() to check its metadata such as size and modification time. If these values differ from what is stored in the index, it means the file has been modified.

We can also recompute the hash of the file and compare it with the stored hash for more accuracy.

If any tracked file is modified, deleted, or new files are present, we consider the working directory dirty and should prevent checkout to avoid losing changes.

Q5.3 — Detached HEAD:

Detached HEAD means that HEAD points directly to a commit hash instead of a branch.

In this state, if we make new commits, they are created but not attached to any branch. This means they can be lost if we move away from them.

To recover such commits, we can create a new branch pointing to that commit hash, or use the commit hash from the log before it gets removed by garbage collection.

Q6.1 — Garbage Collection Algorithm:

To perform garbage collection, we first find all reachable objects starting from branch references.

We begin from each branch head and traverse all commits using parent pointers. From each commit, we get its tree, and from the tree we get all blobs and subtrees. We can use a DFS or BFS traversal for this.

We store all visited object hashes in a set. After traversal, any object not present in this set is considered unreachable and can be deleted.

For a repository with 100,000 commits and multiple branches, we may need to visit all commits, trees, and blobs, which can be a large number of objects.

Q6.2 — GC Race Condition:

Running garbage collection during a commit is dangerous because a commit may be in progress and some objects may not yet be fully referenced.

If GC runs at the same time, it may delete these objects thinking they are unused. This can corrupt the repository.

Git avoids this by using locking mechanisms and ensuring that objects are safely written and referenced before GC runs. It may also use temporary references to protect new objects during the process.